ENTERPRISE RAG KNOWLEDGE SYSTEM FOR INTELLIGENT DOCUMENT QUESTION ANSWERING

MINI PROJECT SYNOPSIS

Submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering – AIML

Submitted to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, UTTAR PRADESH, LUCKNOW

Submitted by

Student Name: Krish Ratudi

Roll No: 2401531530065

Section: 3F

Under the Supervision of

Mr. Yogendra Kumar Vishwakarma

Assistant Professor

Department of Computer Science & Engineering

Department of Computer Science & Engineering

Lloyd Institute of Engineering & Technology

Plot No. 3, Knowledge Park II, Greater Noida, Uttar Pradesh 201306

Session: 2026–27

INDEX

Sr. No.	Topic	Page No.
01	Introduction	3–4
02	Statement of Problem with Objectives	5
03	Literature Survey	6
04	Expected Outcome / Scope of the Project	7
05	Tentative Work Plan (mapped to the 10-week timeline)	8
06	Software / Hardware Required for the Development of the Project	9
07	References	10

Student's Signature Guide's Signature

1. INTRODUCTION

The rapid growth of digital information has made organisational knowledge increasingly difficult to manage and use effectively. Enterprise documents such as policies, manuals, reports, project records, technical documents, frequently asked questions, and internal guidelines contain valuable information, but users often have to search through many files to locate a specific answer. Traditional keyword-based search can return large numbers of results without understanding the meaning of the query or the context in which the information is required.

The Enterprise RAG Knowledge System is proposed as an intelligent document question-answering system based on Retrieval-Augmented Generation (RAG). RAG combines information retrieval with a Large Language Model (LLM). Instead of depending only on information learned during model training, the system first retrieves relevant passages from a controlled knowledge base and then provides those passages to the language model as context for answer generation. This approach is particularly useful when the required information is contained in private, domain-specific, or frequently changing documents.

The proposed system will allow authorised users to upload or access a collection of enterprise documents and ask natural-language questions about their contents. The documents will be processed through an ingestion pipeline. Text will be extracted, cleaned, divided into smaller chunks, converted into numerical vector representations called embeddings, and stored in a vector database. When a user submits a question, the question will also be embedded and used to retrieve semantically relevant chunks. The retrieved context will then be supplied to the LLM through a controlled prompt so that the generated response is grounded in the available documents.

The project domain lies at the intersection of Artificial Intelligence, Natural Language Processing, Information Retrieval, and Generative AI. Important technical concepts include Large Language Models, embeddings, vector databases, semantic search, document chunking, similarity search, retrieval, prompt engineering, context grounding, and hallucination reduction.

A major objective of the system is to make answers more traceable. The proposed interface can display the source document and relevant page or chunk information along with the answer, where available. This helps users verify the response against the original enterprise knowledge instead of treating the generated answer as an independent source of truth.

1. INTRODUCTION (CONTINUED)

The overall workflow of the proposed Enterprise RAG Knowledge System consists of five major stages: document ingestion, preprocessing and chunking, embedding and indexing, retrieval, and grounded response generation. During ingestion, supported documents such as PDF files are loaded into the system. The extracted text is cleaned and divided into manageable chunks with controlled overlap so that useful context is retained across chunk boundaries.

In the indexing stage, each chunk is transformed into an embedding using a sentence-level embedding model. The embeddings are stored in a vector store such as ChromaDB, which supports efficient similarity-based retrieval. At query time, the user's question is converted into an embedding and compared with the indexed document vectors. The most relevant chunks are selected and may be ranked using a retrieval strategy such as Maximum Marginal Relevance (MMR) to improve diversity and reduce redundant context.

The selected passages are inserted into a prompt together with instructions that require the language model to answer using the supplied context. If the retrieved information is insufficient, the system should avoid inventing unsupported facts and instead indicate that the available knowledge base does not provide enough information. This grounded generation mechanism is an important characteristic of the proposed system because enterprise users generally require answers that are relevant, explainable, and connected to source material.

The implementation is planned using Python as the primary programming language. The proposed application stack includes LangChain for retrieval-oriented application orchestration, PyPDF-based document processing, a text-splitting component for chunking, a Hugging Face sentence-transformer model such as all-MiniLM-L6-v2 for embeddings, ChromaDB for vector storage, and a Groq-hosted language model for response generation. Streamlit may be used to provide a simple interactive user interface. The project will be developed in a Python virtual environment and configured through environment variables so that API credentials are not hard-coded in source files.

The system will also be evaluated from both retrieval and generation perspectives. Important evaluation factors include retrieval relevance, answer correctness, source grounding, hallucination behaviour, response latency, and the effect of parameters such as chunk size, chunk overlap, number of retrieved chunks, and retrieval strategy. The project therefore demonstrates not only how to build a RAG pipeline but also how to evaluate and improve an AI application using measurable criteria.

2. STATEMENT OF PROBLEM AND OBJECTIVES

Existing enterprise information systems commonly depend on folder structures, document repositories, and keyword-based search. Such systems are useful for locating files but do not always provide direct answers to natural-language questions. Users may need to open several documents, inspect multiple pages, and manually combine information. In addition, conventional keyword search can fail when the wording of the query differs from the wording used in the source document.

General-purpose Large Language Models can generate fluent answers, but when they are not connected to the organisation's current knowledge base, they may not know private information or may produce unsupported statements. This creates a need for a system that combines the language understanding capabilities of an LLM with controlled retrieval from an enterprise-specific knowledge base.

The proposed Enterprise RAG Knowledge System aims to address these limitations through the following objectives:

- To design and implement an end-to-end Retrieval-Augmented Generation pipeline for enterprise document question answering.
- To develop a document ingestion and preprocessing workflow that extracts text from PDF documents and divides it into meaningful, retrievable chunks.
- To generate semantic embeddings for document chunks and store them in a vector database for efficient similarity-based retrieval.
- To retrieve relevant evidence for a user query and provide the retrieved context to an LLM for grounded answer generation.
- To provide source-aware responses so that users can identify the document and page or chunk supporting an answer, where source metadata is available.
- To reduce unsupported or hallucinated responses by enforcing context-grounded prompting and evaluating answers against retrieved evidence.
- To evaluate the effect of retrieval parameters such as chunk size, overlap, top-k retrieval, and MMR on answer relevance and system performance.
- To develop a simple interactive interface through which users can upload documents and ask questions in natural language.

3. LITERATURE SURVEY

Retrieval-Augmented Generation combines a neural language generator with an external, non-parametric knowledge source so that responses are grounded in retrievable evidence rather than model memory alone. Gao et al. surveyed the evolution of RAG architectures, distinguishing Naive RAG, Advanced RAG, and Modular RAG, and examined the retrieval, generation, and augmentation techniques that make up a typical RAG pipeline. This survey provides the architectural foundation on which the proposed system is designed.

Fan et al. reviewed how RAG is being integrated with modern Large Language Models, covering retrieval strategies, prompt augmentation, and the practical trade-offs involved in combining a retriever with an LLM-based generator. Their survey highlights that RAG performance depends jointly on retrieval quality and how retrieved context is presented to the generator, which directly motivates the proposed system's use of MMR-based retrieval and a controlled prompt template.

Evaluating RAG systems is difficult because both the retrieval and generation components must be assessed together. Es et al. introduced RAGAs, a framework for reference-free evaluation of RAG pipelines using metrics such as faithfulness, answer relevance, and context relevance, removing the need for manually annotated ground-truth answers. This framework is directly relevant to the proposed project's evaluation stage, where similar dimensions are used to judge answer quality and grounding.

Yu et al. surveyed RAG evaluation methods and benchmarks more broadly, comparing metrics for retrieval relevance, answer correctness, and faithfulness across existing evaluation frameworks, and noted the current lack of standardised benchmarks in the field. Their analysis informs the evaluation criteria adopted in the proposed project. In the proposed project, ChromaDB is considered as the vector-store component because it provides a convenient interface for storing embeddings and metadata.

Recent research and surveys on Retrieval-Augmented Generation have examined the complete RAG pipeline, including indexing, retrieval, generation, evaluation, and methods for improving factuality and relevance. These studies indicate that RAG performance depends not only on the selected LLM but also on document preprocessing, chunking strategy, embedding quality, retrieval configuration, and prompt construction.

The proposed project applies these ideas to an enterprise-oriented knowledge system. Rather than training a language model from scratch, the project focuses on building a practical retrieval and generation pipeline that can work with a changing collection of documents. The system will additionally consider source attribution and empirical evaluation, which are important for practical use of generative AI in knowledge-intensive environments.

4. EXPECTED OUTCOME / SCOPE OF THE PROJECT

The expected outcome of the project is a working prototype of an Enterprise RAG Knowledge System capable of answering natural-language questions using information retrieved from an uploaded enterprise document collection.

Key expected features include:

- PDF document ingestion and text extraction.
- Text cleaning, chunking, and metadata preservation.
- Semantic embedding generation for document chunks.
- Vector-based indexing and similarity retrieval using a vector database.
- Query processing and retrieval of the most relevant knowledge chunks.
- Context-grounded response generation through a Large Language Model.
- Source document and page/chunk references for improved answer traceability.
- An interactive interface for document upload and question answering.
- Basic evaluation of retrieval relevance, answer quality, hallucination behaviour, and response latency.

The intended users include students, academic staff, project teams, and organisations that need to search and question collections of technical or administrative documents. Potential applications include internal policy assistants, technical documentation assistants, academic document assistants, project knowledge bases, and frequently asked question systems.

The initial scope will focus on document-based question answering, particularly PDF documents. The prototype is intended to demonstrate the RAG architecture rather than provide a complete enterprise production platform. Security, authentication, large-scale distributed deployment, fine-grained access control, and integration with proprietary enterprise repositories may be considered as future extensions.

Future improvements may include support for multiple document formats, hybrid keyword-plus-vector retrieval, re-ranking models, conversational memory, role-based access control, document versioning, multilingual retrieval, automated evaluation, monitoring, and deployment as a scalable web service. The system could also be extended to process structured data and connect to enterprise knowledge repositories.

5. TENTATIVE WORK PLAN (MAPPED TO THE 10-WEEK TIMELINE)

The proposed project will be completed through a structured ten-week development plan covering analysis, study, design, implementation, testing, documentation, and final presentation.

Week	Major Activity	Planned Work
Week 1	Requirement analysis and problem definition	Define use cases, document types, functional requirements, constraints, and evaluation goals.
Week 2	Literature study and technology selection	Study RAG, embeddings, vector databases, retrieval methods, LLMs, and select the implementation stack.
Week 3	System architecture and data pipeline design	Design ingestion, preprocessing, chunking, embedding, vector storage, retrieval, and generation modules.
Week 4	Document ingestion and preprocessing	Implement PDF loading, text extraction, cleaning, metadata handling, and chunking.
Week 5	Embedding and vector database	Generate embeddings, create the vector collection, store chunks and metadata, and validate indexing.
Week 6	Retriever and RAG chain	Implement query embedding, similarity/MMR retrieval, prompt construction, and grounded LLM response generation.
Week 7	User interface and source attribution	Develop the interactive interface and display retrieved sources/page information with generated answers.
Week 8	Testing and evaluation	Test different chunk sizes, overlap, top-k values, retrieval strategies, latency, relevance, and hallucination behaviour.
Week 9	Refinement and documentation	Fix defects, improve prompts/retrieval, prepare architecture diagrams, screenshots, results, and project documentation.
Week 10	Final validation and presentation	Complete final testing, synopsis/report, demonstration, presentation slides, and project viva preparation.

6. SOFTWARE / HARDWARE REQUIRED FOR THE DEVELOPMENT OF THE PROJECT

The proposed system can be developed on a standard modern laptop or desktop computer. The following resources are planned for implementation and testing.

Category	Requirement
Operating System	Ubuntu Linux / compatible Windows or Linux environment
Programming Language	Python 3.11
Development Environment	Visual Studio Code or equivalent Python IDE
Frameworks / Libraries	LangChain, LangChain Text Splitters, PyPDF, sentence-transformers, Streamlit
Embedding Model	Hugging Face sentence-transformer such as all-MiniLM-L6-v2
Vector Database	ChromaDB
LLM / Inference	Groq API with a supported instruction/chat model
Configuration	Python virtual environment and .env-based configuration for API credentials
Version Control	Git and GitHub
Document Input	PDF documents containing enterprise or project knowledge
Processor	Modern multi-core Intel/AMD processor or equivalent
RAM	Minimum 8 GB; 16 GB recommended for smoother development
Storage	At least 10 GB free space for project files, packages, vector data, and test documents
Internet	Required when using cloud-hosted LLM/API services and downloading models/packages

The exact hardware requirement may vary depending on the embedding model, document collection size, and whether inference is performed locally or through an API. The proposed prototype is designed so that the main language-model computation can be accessed through an API, reducing the need for a high-end local GPU.

7. REFERENCES

ll four references are now from the last 3 years (2023–2024):

1. **Gao et al., 2023** — RAG survey (Naive/Advanced/Modular RAG architectures)
2. **Es et al., 2024** — RAGAs, an automated RAG evaluation framework
3. **Yu et al., 2024** — Survey on RAG evaluation methods and benchmarks
4. **Fan et al., 2024** — Survey on RAG meeting LLMs (KDD 2024)